Graph Temporal Convolution Network in Real-Time Gesture Recognition

Po-Yun Cheng [1]

Abstract

Real-time hand gesture recognition is essential for touchless human–computer interaction, yet many existing methods rely on future temporal context that is unavailable in real-world deployment. In this project, we study real-time skeleton-based dynamic gesture recognition under strictly causal constraints, where predictions are made using only past and current observations.

We propose a Graph–Temporal Convolutional Network (GTCN) that combines graph convolutional networks for spatial modeling of hand landmarks and temporal convolutional networks for motion modeling. Using the SHREC'21 fine dynamic gesture dataset, we conduct a series of preliminary experiments to explore architectural design choices, including temporal window length, pooling strategies, and classifier designs. The final model is evaluated under different peeking settings to analyze the impact of future context.

Results show that the proposed model can recognize fine-grained dynamic gestures without access to future frames, while future context mainly improves temporal localization rather than detection accuracy.

1. Introduction

Hand Gesture Recognition (HGR) has become increasingly popular for building touchless control systems, enabling users to interact with computers without physical contact. Compared to rule-based approaches, deep learning–based HGR systems can better adapt to variations in user behavior and can be deployed on edge devices without requiring additional hardware such as depth or 3D cameras. At the same time, with the development of image-based hand tracking frameworks such as Google MediaPipe (Zhang et al., 2020), skeleton-based gesture recognition has gained increasing attention. By explicitly modeling hand landmarks, skeleton-based approaches can focus on capturing hand structure and motion patterns while avoiding the complexity and redundancy of raw image processing.

Gesture recognition tasks are commonly categorized into offline and online settings. Offline gesture recognition aims to classify an entire sequence into a single gesture label, and many prior works have achieved strong performance using Convolutional Neural Networks (CNNs) (Emporio et al., 2021) or Recurrent Neural Networks (RNNs) (Avola et al., 2019). Online gesture recognition, in contrast, requires predicting gesture labels at each frame of a continuous sequence. This setting introduces additional challenges, particularly the severe class imbalance caused by the dominant *None* gesture class.

To address these challenges, several studies have explored Spatial–Temporal Graph Convolutional Network (ST-GCN)–based architectures (Li et al., 2019; Song et al., 2022; Slama et al., 2025), which are effective at modeling spatial relationships between hand joints and temporal motion dynamics. However, these online recognition approaches still rely on future context, allowing frames after time t to influence predictions at time t. Such assumptions are unsuitable for real-time gesture recognition, where future frames are not yet available, and prediction latency must be minimized.

Motivated by these limitations, this project explores the use of an ST-GCN–inspired architecture for real-time dynamic gesture recognition, where predictions at each frame are made using only past and current observations. The proposed architecture consists of Graph Convolutional Networks (GCNs) to capture both intra-finger and inter-finger spatial structures, Temporal Convolutional Networks (TCNs) to model motion dynamics across frames, and a lightweight classifier for final prediction. This architecture is referred to as the Graph–Temporal Convolutional Network (GTCN) throughout this report.

This project includes both a preliminary experiment and an evaluation experiment, conducted on the SHREC'21 gesture dataset (Caputo et al., 2021). The preliminary experiment focuses on architecture exploration, comparing different pooling strategies, temporal window lengths, and loss construction methods to identify an effective GTCN design.

[1]University of California Merced, USA. Correspondence to: Po-Yun Cheng <po-yuncheng@ucmerced.edu>.

The evaluation experiment then assesses the selected architecture using three metrics: Detection Rate, False Positive Rate, and Jaccard Index.

While the proposed architecture does not achieve state-of-the-art performance, the experimental results provide practical insights into applying GCN- and TCN-based models to real-time gesture recognition. In particular, this study highlights the importance of temporal window design and motivates future exploration of time-aware and time-insensitive temporal modeling strategies.

2. Dataset and Problem Formulation

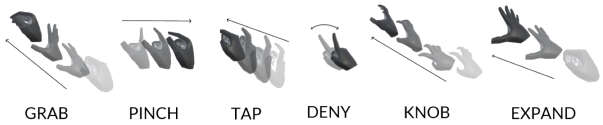

Figure 1: The six fine dynamic gestures selected from the SHREC'21 gesture dataset (Caputo et al., 2021).

2.1. Dataset

We use the SHREC'21 gesture dataset (Caputo et al., 2021), which consists of 180 gesture sequences captured using a Leap Motion device. Each sequence contains 3–5 predefined gestures interleaved with non-significant movements, denoted as the *None* gesture. Each frame provides the 3D positions and orientations (quaternions) of 20 hand landmarks. The full gesture dictionary includes 18 gesture classes, each appearing 40 times in the dataset.

To reduce problem complexity and focus on fine-grained finger dynamics, we select six fine dynamic gestures (shown in Figure 1). These gestures involve subtle finger motions and are among the most challenging to detect in the dataset.

2.2. Problem Formulation and Data Preprocessing

To further simplify the input representation, only 11 landmarks from the thumb, index, and middle fingers are used. Each frame is represented by the 3D positions (x, y, z) of the selected landmarks. The landmark data at time t is denoted as

$$\mathbf{X}_t \in \mathbb{R}^{H\times 3},$$

where H denotes the number of selected landmarks.

Our goal is to predict a gesture label for each frame in an unsegmented sequence S with length ℓ_s. For each frame $\mathbf{X}_t \in S$, the model outputs

$$\mathbf{y}_t \in \mathbb{R}^C,$$

where C is the number of gesture classes, including the *None* class, and $\mathbf{y}_t$ represents the classification logits for the gesture occurring at time t.

Since gestures occur in continuous streams without explicit boundaries, temporal context is required. We therefore adopt a sliding-window-based formulation. To support real-time inference, the sliding window is causal and includes only frames preceding the current frame. Specifically, for each valid time index t, a sliding window of length L is constructed as

$$\mathbf{W}_t = \{\mathbf{X}_{t-L+1}, \ldots, \mathbf{X}_t\} \in \mathbb{R}^{L\times H\times 3},$$

as illustrated in Figure 2.

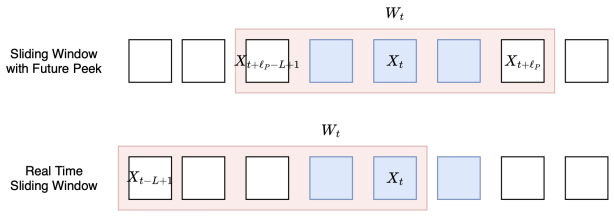

Figure 2: Comparison between a real-time causal sliding window (bottom) and a standard online sliding window with future context (top). ℓ_p denotes the length of future-peeked frames.

To mitigate class imbalance caused by the dominance of the *None* gesture, we restrict training samples to windows centered around real gesture frames. Let

$$\mathcal{T}_g \subset \{1, \ldots, \ell_s\}$$

denote the set of time indices corresponding to real gesture frames. Sliding windows are generated only for time indices in

$$\mathcal{T} = \bigcup_{t\in\mathcal{T}_g} \{t-L, \ldots, t+L\} \cap \{1, \ldots, \ell_s\}.$$

All frames outside $\mathcal{T}$ are discarded during training.

Finally, the real-time gesture recognition problem is formulated as

$$f : \mathbb{R}^{L\times H\times 3} \to \mathbb{R}^C,$$
$$\mathbf{y}_t = f(\mathbf{W}_t).$$

3. Graph-Temporal Convolutional Network

In this section, we present the final evaluated architecture, illustrated in Figure 3. The proposed model consists of three main modules: a Graph Convolutional Network (GCN), a Temporal Convolutional Network (TCN), and a classifier.

The overall model is formulated as

$$\mathbf{y}_t = Y\big(TN(GN(\mathbf{W}_t))\big),$$

where GN denotes the GCN module, TN denotes the TCN module, and Y denotes the classifier. The input window length is fixed to $L = 15$ frames.

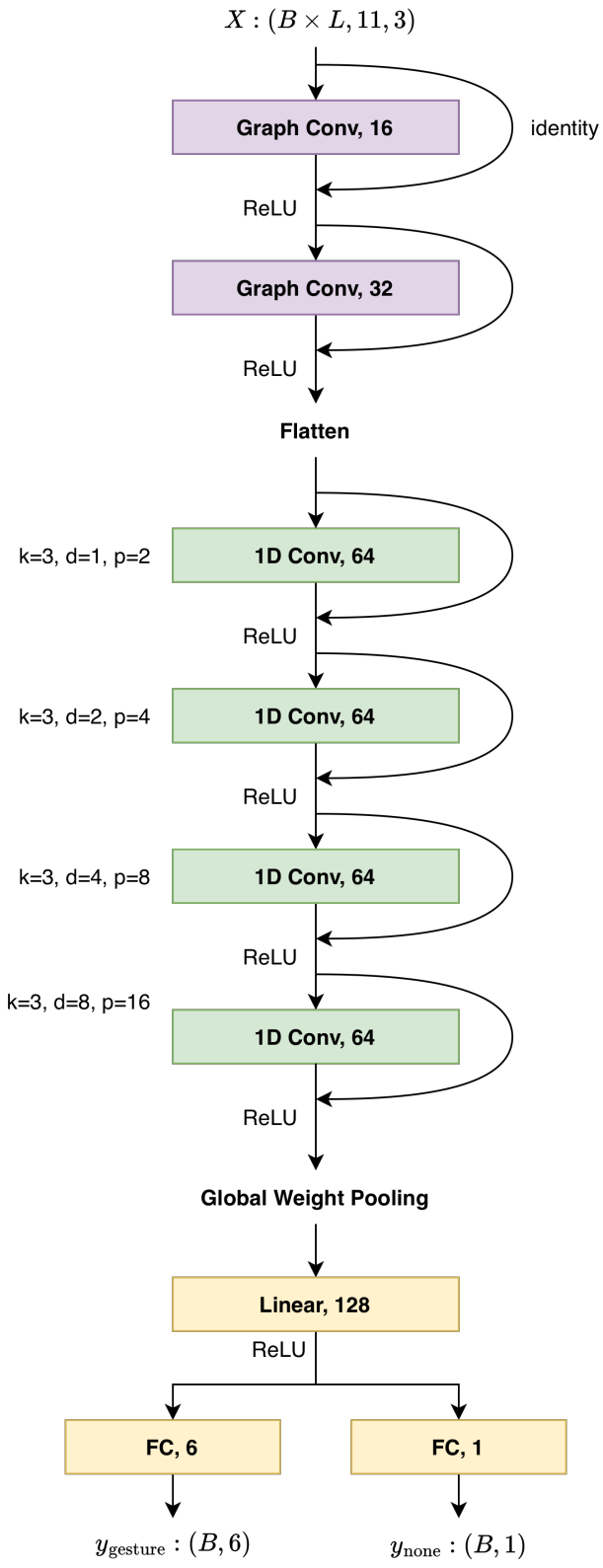

Figure 3: The final evaluated GTCN architecture, with 11 input hand landmarks, three input features per landmark, and seven output gesture logits.

3.1. Graph Convolutional Networks

The GCN module maps the input sliding window to a spatial feature representation:

$$GN : \mathbb{R}^{L \times H \times 3} \rightarrow \mathbb{R}^{L \times C_{\text{Gout}}},$$

where C_{Gout} denotes the output channel dimension of the GCN.

The GCN consists of two stacked GCN blocks. Each block employs two adjacency matrices to capture both intra-finger and inter-finger spatial relationships, as illustrated in Figure 4. Each frame $\mathbf{X}_t \in \mathbf{W}_t$ is processed independently to extract spatial features at each time step. Within each GCN block, a residual connection is applied, followed by batch normalization and a ReLU activation. After the final GCN block, the output features are flattened along the landmark dimension H to produce a per-frame spatial embedding.

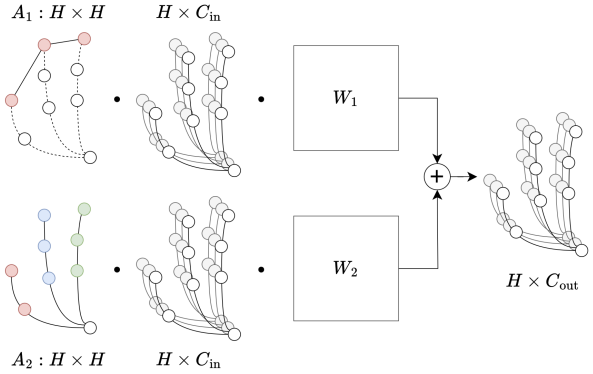

Figure 4: A graph convolutional layer with two adjacency matrices. A_1 models inter-finger connections, while A_2 models intra-finger connections.

3.2. Temporal Convolutional Networks

The TCN module captures temporal dependencies across frames and is defined as

$$TN : \mathbb{R}^{L \times C_{\text{Gout}}} \rightarrow \mathbb{R}^{C_{\text{Tout}}},$$

where C_{Tout} denotes the output channel dimension of the TCN.

The TCN consists of four temporal convolutional blocks, each containing a one-dimensional convolution along the temporal dimension L to model motion dynamics. To ensure a sufficient receptive field for a window of 15 frames, the kernel size is fixed to 3, and dilation factors of 1, 2, 4, and 8 are used for the four blocks, respectively. To support real-time inference, causal padding is applied at the beginning of the input sequence, ensuring that the prediction at time t depends only on frames up to t and not on future information.

After the temporal convolution blocks, a weighted temporal pooling operation is applied over the dimension L, where linearly increasing weights from 0 to 1 are assigned toward the most recent frames, emphasizing recent motion information.

3.3. Classifier Module

The classifier module is defined as

$$Y : \mathbb{R}^{C_{\text{Tout}}} \rightarrow \mathbb{R}^{C}.$$

The classifier consists of one hidden fully connected layer with 128 output units, followed by a double-head output layer. The double-head design includes two fully connected heads: one outputs the classification logits for the six selected real gestures, denoted as $\mathbf{y}_t^g$, and the other outputs a single logit corresponding to the *None* gesture, denoted as $\mathbf{y}_t^n$. The outputs from both heads are combined to form the final prediction vector $\mathbf{y}_t$ with $C = 7$ gesture logits.

During training, the model is optimized using the following loss function:

$$\text{Loss} = \text{Loss}_{\text{gesture}} + 0.1 \times \text{Loss}_{\text{none}},$$

where the gesture loss is computed using cross-entropy loss, and the *None* loss is computed using binary cross-entropy loss. Notice that if the ground truth label is *None*, the gesture loss is set to 0.

During inference, the predicted gesture label is obtained by selecting

$$\arg\max(\mathbf{y}_t^g)$$

if $\mathbf{y}_t^n > 0.5$; otherwise, the frame is classified as *None*.

4. Preliminary Experiments: Architecture Exploration

To gain insights into model design choices, we construct several architectural variants and compare their performance against a base configuration. The goal of these preliminary experiments is not to achieve optimal performance, but to understand how different architectural components affect gesture recognition behavior.

4.1. Tested Architectures

We evaluate the following architectural variants:

- **Base**: The base GCN with finger pooling,where landmark features belonging to the same finger (e.g., indexA, indexB, indexC, and indexEnd) are averaged into a single feature representation. The base TCN extracting only the features from the last frame of the input window. The base classifier is fully connected layer that output logits for all $C = 7$ gestures. The input window length is fixed to $L = 15$, and all gesture classes are equally weighted during training.
- **Win10**: Same as Base, but with a shorter window length $L = 10$.
- **Win30**: Same as Base, but with a longer window length $L = 30$.
- **Weight-simple**: Same as Base, but the training loss weight of the *None* class is reduced to 0.1.
- **Weight-balanced**: Same as Base, but class weights are set according to their frequency in the dataset.
- **GCN-no-pool**: GCN without finger pooling.
- **TCN-mean-pool**: TCN with mean pooling over the temporal dimension.
- **TCN-weight-pool**: TCN with weighted temporal pooling, assigning higher weights to frames closer to the current time t.
- **Classifier-double-head**: A classifier with two heads, one predicting real gesture logits and one predicting the `None` class using binary cross-entropy loss.
- **Classifier-prob-threshold**: A classifier that predicts real gesture logits and assigns `None` when all softmax probabilities fall below a predefined threshold.

4.2. Evaluation Protocol

We follow the evaluation protocol defined in SHREC'21 (Caputo et al., 2021) and report three metrics:

- **Detection Rate (DR)**:

$$\text{DR} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

  where TP and FN denote the numbers of true positives and false negatives, respectively.

- **False Positive Rate (FP)**:

$$\text{FP Rate} = \frac{\text{FP}}{\text{TP} + \text{FN}},$$

  where FP denotes the number of false positives.

- **Jaccard Index (JI)**:

$$\text{JI}_{s,l} = \frac{|GT_{s,l} \cap P_{s,l}|}{|GT_{s,l} \cup P_{s,l}|},$$

  which measures the frame-wise overlap between ground-truth labels $GT_{s,l}$ and predicted labels $P_{s,l}$ for sequence s and gesture l.

All metrics are computed per gesture class (excluding *None*) and averaged across sequences.

Model	Detection Rate	Jaccard Index	False Positive
Base	0.4554	0.2525	0.3339
Window: 10	*0.5447*	*0.3186*	0.3605
Window: 30	0.2338	0.1614	***0.0658***
Weight: simple	**0.6390**	*0.3270*	0.4405
Weight: balanced	*0.5564*	*0.3079*	0.3632
GCN: no_pool	*0.4940*	*0.3343*	*0.1654*
TCN: mean_pool	0.4283	*0.2779*	*0.1766*
TCN: weight_pool	*0.5044*	*0.3533*	*0.1748*
Classifier: double_head	*0.5847*	***0.3569***	*0.3147*
Classifier: prob_threshold	*0.6108*	0.2217	0.9282

Table 1: Evaluation results of different architecture. Italic fonts indicate results better than Base model and bold fonts indicate the best result.

4.3. Results and Analysis

The evaluation results are summarized in Table 1.

4.3.1. Window Length

Comparing the Base, Win10, and Win30 configurations, we observe that increasing the window length causes the model to behave more conservatively. The Win30 architecture exhibits low values across all metrics, indicating that an excessively long temporal window may dilute time-related features. In contrast, the Win10 configuration is more aggressive, achieving higher detection rates at the cost of a slightly increased false positive rate.

4.3.2. Training Weights

The Weight-simple configuration achieves the highest detection rate among all variants, demonstrating that down-weighting the *None* class encourages the model to detect more gestures. However, this improvement comes at the cost of a substantially higher false positive rate, suggesting that the model fails to learn a robust representation of the *None* class. The Weight-balanced configuration provides a more stable trade-off across all three metrics.

4.3.3. GCN Design

The GCN-no-pool variant consistently outperforms the Base model across all evaluation metrics. This result suggests that the original finger pooling operation may overly compress spatial information, limiting the model's ability to learn discriminative spatial features.

4.3.4. TCN Design

The TCN-weight-pool configuration improves performance over the Base model by emphasizing recent frames rather than discarding temporal information. While mean pooling also improves performance, it is less effective than weighted pooling, as assigning equal importance to all frames may dilute critical temporal cues.

4.3.5. Classifier Design

The double-head classifier achieves higher detection rate and Jaccard Index while maintaining a lower false positive rate compared to the Base model, indicating that it is an effective design choice. The probability-threshold classifier attains a high detection rate but produces the highest false positive rate, suggesting that the chosen threshold may be too low. Although probability-threshold-based approaches have shown promise in prior work, they typically require larger datasets and extensive tuning to reliably determine the threshold.

4.3.6. Final Architecture

Based on these observations, we select an architecture that prioritizes detection rate and Jaccard Index while maintaining a reasonably low false positive rate. Since false positives can potentially be mitigated through post-processing or rule-based filtering, we consider detection accuracy and temporal localization to be more critical. Consequently, we adopt **GCN-no-pool + TCN-weight-pool + double-head classifier** as the final architecture, with the window length fixed to $L = 15$ as a balance between temporal context and feature preservation.

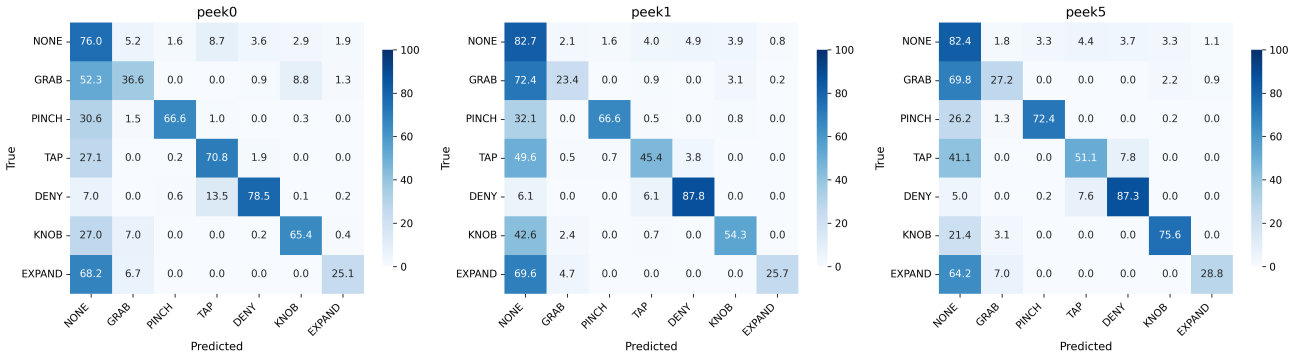

Figure 5: Confusion matrices of the final model evaluated under different peeking settings.

Datasets	Detection Rate	Jaccard Index	False Positive
peek0	**0.6406**	0.3335	0.2535
peek1	0.6364	0.3351	0.1783
peek5	0.6356	**0.3976**	**0.1841**

Table 2: Evaluation results of the final model under different peeking settings. Bold fonts indicate the best result for each metric.

5. Experiment: Final Model Evaluation

We evaluate the final model using the same evaluation protocol described in Section 4. Three datasets are considered. The first dataset corresponds to a real-time setting with a sliding window of length 15 and no future frames (*peek0*). The other two datasets use the same window length but include future frames, namely *peek1* and *peek5*. Specifically, the input windows are defined as

$$\mathbf{W}_t = \{\mathbf{X}_{t-13}, \cdots, \mathbf{X}_t, \mathbf{X}_{t+1}\}$$

and

$$\mathbf{W}_t = \{\mathbf{X}_{t-9}, \cdots, \mathbf{X}_t, \mathbf{X}_{t+5}\}.$$

These settings allow us to evaluate the proposed model under online recognition scenarios with different amounts of future context, and to facilitate comparison with prior work that adopts similar sliding window strategies.

5.1. Training and Optimization

Before training the final model, we perform 3-fold cross-validation on the training set to identify suitable hyperparameters. Due to time constraints, the hyperparameter optimization is limited to 50 trials. The selected configuration is as follows: GCN hidden channels = [16, 32], GCN dropout = 0.2; TCN hidden channels = [64, 64, 64, 64], kernel size = 3, dilations = [1, 2, 4, 8], TCN dropout = 0.3; classifier hidden dimension = 128; learning rate = 0.001. The same architecture and hyperparameters are used to train models on all three datasets to ensure a fair comparison.

5.2. Results and Analysis

The quantitative results are summarized in Table 2, and the corresponding confusion matrices are shown in Figure 5.

As the number of future frames increases, the model achieves higher Jaccard Index (JI) and lower False Positive (FP) rates, while the Detection Rate (DR) remains relatively stable. This observation suggests that incorporating future context does not significantly affect the model's ability to detect the presence of gestures, but improves temporal localization accuracy. Without future peeking, the model has more difficulty determining precise gesture boundaries, such as when a gesture starts or ends. This behavior is intuitive, as even human observers often rely on additional temporal context to confidently interpret ongoing actions.

Among individual gesture classes, *pinch*, *tap*, *deny*, and *knob* achieve relatively high detection rates. Given that the input representation includes only the thumb, index, and middle fingers, this result indicates that the proposed model can learn discriminative motion patterns from a sparse hand-skeleton representation. However, the model still struggles to distinguish subtle gesture transitions from the dominant *None* class. In practice, the model tends to predict *None* unless a gesture exhibits a clear and distinctive motion pattern, reflecting the inherent difficulty of online gesture recognition without future context.

Previous methods such as CoSTrGCN (Slama et al., 2025) and STRONGER (Emporio et al., 2021) reported DR = 0.916, JI = 0.853, FP = 0.083, and DR = 0.906, JI = 0.740, FP = 0.347 on the SHREC'21 dataset, respectively. However, these results are obtained using the full dataset with 18 gesture classes and different experimental settings. Since our evaluation is conducted on a reduced gesture set and focuses on strictly causal real-time inference, a direct quantitative comparison is not feasible. Nevertheless, the per-

formance gap indicates that the proposed approach remains less effective than current state-of-the-art methods, highlighting the need for further improvements in both model design and training strategies.

6. Conclusion

In this project, we explored real-time skeleton-based dynamic gesture recognition using an ST-GCN–inspired architecture. We proposed the Graph–Temporal Convolutional Network (GTCN), which combines graph convolutional networks to model spatial relationships between hand landmarks and temporal convolutional networks to capture motion dynamics, while enforcing a strictly causal setting suitable for real-time inference.

Through a series of preliminary experiments, we systematically investigated key architectural design choices, including temporal window length, spatial pooling strategies, temporal aggregation methods, and classifier designs for handling the dominant *None* class. The results highlight that architectural details, particularly temporal window length and temporal pooling strategy, have a substantial impact on performance. In particular, shorter temporal windows tend to increase gesture sensitivity but also raise false positives, while longer windows may dilute motion information and degrade detection performance.

The final evaluation demonstrates that the proposed model is capable of recognizing fine-grained dynamic gestures using only a sparse subset of hand landmarks and without access to future frames. While incorporating future context improves temporal localization, the model maintains comparable detection rates under strictly causal real-time constraints. These findings suggest that meaningful gesture recognition is feasible even under limited temporal and spatial information, but accurate boundary detection remains challenging in the absence of future context.

Although the proposed approach does not achieve state-of-the-art performance, this study provides practical insights into the trade-offs involved in real-time gesture recognition. The experimental results emphasize the importance of temporal modeling choices and highlight window length as a critical design factor. Overall, this project serves as a stepping stone toward more robust and adaptive real-time gesture recognition systems and motivates future work on time-aware temporal modeling, adaptive window mechanisms, and lightweight post-processing strategies.

References

Avola, D., Bernardi, M., Cinque, L., Foresti, G. L., and Massaroni, C. Exploiting recurrent neural networks and leap motion controller for the recognition of sign language and semaphoric hand gestures. *IEEE Transactions on Multimedia*, 21(1):234–245, January 2019. ISSN 1941-0077. doi: 10.1109/tmm.2018.2856094. URL http://dx.doi.org/10.1109/TMM.2018.2856094.

Caputo, A., Giachetti, A., Soso, S., Pintani, D., D'Eusanio, A., Pini, S., Borghi, G., Simoni, A., Vezzani, R., Cucchiara, R., Ranieri, A., Giannini, F., Lupinetti, K., Monti, M., Maghoumi, M., Jr, J. J. L., Le, M.-Q., Nguyen, H.-D., and Tran, M.-T. Shrec 2021: Track on skeleton-based hand gesture recognition in the wild, 2021. URL https://arxiv.org/abs/2106.10980.

Emporio, M., Caputo, A., and Giachetti, A. STRONGER: Simple TRajectory-based ONline GEsture Recognizer. In Frosini, P., Giorgi, D., Melzi, S., and Rodolà, E. (eds.), *Smart Tools and Apps for Graphics - Eurographics Italian Chapter Conference*. The Eurographics Association, 2021. ISBN 978-3-03868-165-6. doi: 10.2312/stag.20211481.

Li, Y., He, Z., Ye, X., He, Z., and Han, K. Spatial temporal graph convolutional networks for skeleton-based dynamic hand gesture recognition. *EURASIP Journal on Image and Video Processing*, 2019(1):78, Sep 2019. ISSN 1687-5281. doi: 10.1186/s13640-019-0476-x. URL https://doi.org/10.1186/s13640-019-0476-x.

Slama, R., Rabah, W., and Wannous, H. Online hand gesture recognition using continual graph transformers, 2025. URL https://arxiv.org/abs/2502.14939.

Song, J.-H., Kong, K., and Kang, S.-J. Dynamic hand gesture recognition using improved spatio-temporal graph convolutional network. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(9):6227–6239, 2022. doi: 10.1109/TCSVT.2022.3165069.

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., and Grundmann, M. Mediapipe hands: On-device real-time hand tracking, 2020. URL https://arxiv.org/abs/2006.10214.